

“Национальный исследовательский
университет ИТМО”
Факультет Мобильные и сетевые технологии

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине “Администрирование компьютерных сетей”

Работу выполнили:

Котков Дмитрий

Дидык Глеб

Кондилеев Егор

Голинский Даня

Санкт-Петербург
2026 г.

Содержание

Цель работы

1. Часть 1. Поднимаем PostgreSQL

1.1. Подготовка Dockerfile

1.2. Описание сервисов в docker-compose.yml

1.3. Конфигурация Patroni

1.4. Запуск кластера и проверка ролей

Ответы на вопросы к части 1

2. Часть 2. Проверяем репликацию

3. Часть 3. Настраиваем высокую доступность через HAProxy

4. Проверка отказоустойчивости (failover)

5. Вывод

Цель работы

Развернуть и настроить высокодоступный кластер PostgreSQL с использованием Docker, Patroni, ZooKeeper и HAProxy. Проверить автоматическое определение лидера, потоковую репликацию данных, маршрутизацию клиентских подключений на ведущую ноду и отказоустойчивость при принудительном выходе из строя текущего лидера.

1. Часть 1. Поднимаем PostgreSQL

1.1. Подготовка Dockerfile

Для работы Patroni необходим доступ к бинарным файлам PostgreSQL, поэтому был подготовлен собственный образ на базе PostgreSQL 15. В образ устанавливаются необходимые зависимости, создается виртуальное окружение Python и устанавливается Patroni с поддержкой ZooKeeper.

```
FROM postgres:15

RUN apt-get update -y && \
    apt-get install -y netcat-openbsd python3-pip curl python3-psycpg2 python3-venv iputils-
ping

RUN python3 -m venv /opt/patroni-venv && \
    /opt/patroni-venv/bin/pip install --upgrade pip && \
    /opt/patroni-venv/bin/pip install patroni[zookeeper] psycpg2-binary

COPY postgres0.yml /postgres0.yml
COPY postgres1.yml /postgres1.yml

ENV PATH="/opt/patroni-venv/bin:$PATH"
USER postgres
```

1.2. Описание сервисов в docker-compose.yml

На первом этапе в Docker Compose описаны две PostgreSQL-ноды и ZooKeeper. Внешние порты PostgreSQL различаются: 5433 для pg-master и 5434 для pg-slave. REST API Patroni на порту 8008 используется внутри Docker-сети.

```
services:
  pg-master:
```

```

build: .
image: localhost/postgres:patroni
container_name: pg-master
hostname: pg-master
restart: always
expose:
  - "8008"
ports:
  - "5433:5432"
volumes:
  - pg-master:/var/lib/postgresql/data
command: patroni /postgres0.yml

pg-slave:
build: .
image: localhost/postgres:patroni
container_name: pg-slave
hostname: pg-slave
restart: always
expose:
  - "8008"
ports:
  - "5434:5432"
volumes:
  - pg-slave:/var/lib/postgresql/data
command: patroni /postgres1.yml

zoo:
image: confluentinc/cp-zookeeper:7.7.1
container_name: zoo
hostname: zoo
restart: always
ports:
  - "2181:2181"
environment:
  ZOOKEEPER_CLIENT_PORT: 2181
  ZOOKEEPER_TICK_TIME: 2000

volumes:
  pg-master:
  pg-slave:

```

Вопрос 1. В чем разница между expose и ports?

Директива `expose` объявляет порт контейнера для использования другими контейнерами внутри одной Docker-сети, но не публикует этот порт на хост-

машине. В нашей конфигурации порт 8008 используется для REST API Patroni и нужен для взаимодействия сервисов внутри Docker-сети. Директива `ports`, напротив, выполняет проброс порта контейнера на порт хост-машины. Например, запись `5433:5432` означает, что PostgreSQL внутри контейнера работает на порту 5432, а с хост-машины к нему можно подключиться через `localhost:5433`. Таким образом, `expose` используется для внутреннего доступа между контейнерами, а `ports` - для публикации сервиса на хосте.

1.3. Конфигурация Patroni

Для первой ноды создан файл `postgres0.yml`. Обе ноды входят в один `scope` `my_cluster` и используют ZooKeeper по адресу `zoo:2181`. Включен синхронный режим, `replication slots` и `pg_rewind`. Для второй ноды `postgres1.yml` изменены имя узла, REST-адрес, `connect_address`, каталог данных и `pgpass`.

```
scope: my_cluster
name: postgresql0

restapi:
  listen: pg-master:8008
  connect_address: pg-master:8008

zookeeper:
  hosts:
    - zoo:2181

bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    synchronous_mode: true
  postgresql:
    use_pg_rewind: true
    use_slots: true
    parameters:
      wal_level: replica
      hot_standby: "on"
      max_wal_senders: 10
      max_replication_slots: 10
```

```

        wal_log_hints: "on"
pg_hba:
  - host replication replicator 0.0.0.0/0 md5
  - host all all 0.0.0.0/0 md5

postgresql:
  listen: 0.0.0.0:5432
  connect_address: pg-master:5432
  data_dir: /var/lib/postgresql/data/postgresql0
  bin_dir: /usr/lib/postgresql/15/bin
  pgpass: /tmp/pgpass0
  authentication:
    replication:
      username: replicator
      password: rep-pass
    superuser:
      username: postgres
      password: postgres

```

1.4. Запуск кластера и проверка ролей

Кластер был собран и запущен командой `docker compose up -d --build`. После запуска проверено состояние контейнеров.

```

lab1 git:(main) docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
pg-master	localhost/postgres:patroni	"docker-entrypoint.s..."	pg-master	59 seconds ago	Up 58 seconds	0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
pg-slave	localhost/postgres:patroni	"docker-entrypoint.s..."	pg-slave	59 seconds ago	Up 58 seconds	0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp
zoo	confluentinc/cp-zookeeper:7.7.1	"/etc/confluent/dock..."	zoo	59 seconds ago	Up 58 seconds	0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp

```

lab1 git:(main)

```

Рисунок 1 - Состояние контейнеров `pg-master`, `pg-slave` и `zoo` после запуска

По логам ZooKeeper установлено, что сервер успешно запущен и принимает подключения на порту 2181. В логах второй PostgreSQL-ноды зафиксированы создание реплики через `basebackup`, переход в `standby mode` и запуск `streaming WAL` от `primary`.

```

pg-slave | 2026-10-06 16:41:24.611 INFO: Zookeeper connection established, state: CONNECTED
pg-slave | 2026-10-06 16:41:24.615 INFO: No PostgreSQL configuration items changed, nothing to reload.
pg-slave | 2026-10-06 16:41:24.616 INFO: REST API thread_pool_size = 5
pg-slave | 2026-10-06 16:41:24.620 INFO: Lock owner: None; I am postgresql1
pg-slave | 2026-10-06 16:41:24.621 INFO: waiting for leader to bootstrap
pg-slave | 2026-10-06 16:41:34.638 INFO: Lock owner: postgresql0; I am postgresql1
pg-slave | 2026-10-06 16:41:34.639 INFO: trying to bootstrap from leader 'postgresql0'
pg-slave | WARNING: skipping special file "/.s.PGSQL.5432"
pg-slave | WARNING: skipping special file "/.s.PGSQL.5432"
pg-slave | 2026-10-06 16:41:35.199 INFO: replica has been created using basebackup
pg-slave | 2026-10-06 16:41:35.199 INFO: bootstrapped from leader 'postgresql0'
pg-slave | 2026-10-06 16:41:35.306 UTC [32] LOG: starting PostgreSQL 15.19 (Debian 15.19-1.pgdg13+2) on aarch64-unknown-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
pg-slave | 2026-10-06 16:41:35.306 UTC [32] LOG: listening on IPv4 address "0.0.0.0", port 5432
pg-slave | 2026-10-06 16:41:35.307 UTC [32] LOG: listening on Unix socket "/.s.PGSQL.5432"
pg-slave | 2026-10-06 16:41:35.308 UTC [35] LOG: database system was interrupted; last known up at 2026-10-06 16:41:35 UTC
pg-slave | 2026-10-06 16:41:35.312 INFO: postmaster pid=32
pg-slave | 2026-10-06 16:41:35.314 UTC [37] FATAL: the database system is starting up
pg-slave | localhost:5432 - rejecting connections
pg-slave | 2026-10-06 16:41:35.316 UTC [39] FATAL: the database system is starting up
pg-slave | localhost:5432 - rejecting connections
pg-slave | 2026-10-06 16:41:35.335 UTC [35] LOG: entering standby mode
pg-slave | 2026-10-06 16:41:35.335 UTC [35] LOG: starting backup recovery with redo LSN 0/2000028, checkpoint LSN 0/2000060, on timeline ID 1
pg-slave | 2026-10-06 16:41:35.336 UTC [35] LOG: redo starts at 0/2000028
pg-slave | 2026-10-06 16:41:35.337 UTC [35] LOG: completed backup recovery with redo LSN 0/2000028 and end LSN 0/2000100
pg-slave | 2026-10-06 16:41:35.337 UTC [35] LOG: consistent recovery state reached at 0/2000100
pg-slave | 2026-10-06 16:41:35.337 UTC [32] LOG: database system is ready to accept read-only connections
pg-slave | 2026-10-06 16:41:35.341 UTC [41] LOG: started streaming WAL from primary at 0/3000000 on timeline 1
pg-slave | localhost:5432 - accepting connections
pg-slave | 2026-10-06 16:41:36.342 INFO: Lock owner: postgresql0; I am postgresql1
pg-slave | 2026-10-06 16:41:36.342 INFO: establishing a new patroni heartbeat connection to postgres
pg-slave | 2026-10-06 16:41:36.354 INFO: no action. I am (postgresql1), a secondary, and following a leader (postgresql0)
pg-slave | 2026-10-06 16:41:46.353 INFO: no action. I am (postgresql1), a secondary, and following a leader (postgresql0)
pg-slave | 2026-10-06 16:41:56.352 INFO: no action. I am (postgresql1), a secondary, and following a leader (postgresql0)

```

Рисунок 2 - Фрагмент логов pg-slave: создание реплики и запуск потоковой WAL-репликации

Командой `patronictl list` проверено текущее состояние кластера. На первоначальном запуске `postgresql0` (`pg-master`) является Leader, а `postgresql1` (`pg-slave`) работает как Sync Standby в состоянии streaming. Lag равен 0.

```

lab1 git:(main) docker exec -it pg-master patronictl -c /postgres0.yml list
+ Cluster: my_cluster (7693595896378900513) -----+-----+-----+-----+
| Member   | Host   | Role   | State   | TL | Receive LSN | Lag | Replay LSN | Lag |
+-----+-----+-----+-----+-----+-----+-----+-----+
| postgresql0 | pg-master | Leader   | running | 1 |             |     |             |     |
| postgresql1 | pg-slave  | Sync Standby | streaming | 1 | 0/304B4C8 | 0 | 0/304B4C8 | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+

```

Рисунок 3 - Начальное состояние кластера Patroni

Вопрос 2. Будет ли образ пересобран при обычном перезапуске Compose? Что произойдет после изменения `postgresX.yml` или `Dockerfile`?

При обычном перезапуске Docker Compose уже существующий образ автоматически заново не собирается: используется ранее созданный image. Если изменить `postgres0.yml` или `postgres1.yml`, изменения также не попадут в уже созданный образ, поскольку эти файлы копируются внутрь него инструкциями COPY из `Dockerfile`. Поэтому после изменения конфигурационных файлов требуется повторная сборка образа. То же самое относится к изменению самого `Dockerfile`. Для применения таких изменений необходимо выполнить `docker compose up -d --build`. При повторной сборке Docker может использовать кэш для слоев, содержимое которых не изменилось.

2. Часть 2. Проверяем репликацию

2.1. Создание тестовых данных на Leader

Для проверки репликации выполнено подключение к текущему лидеру. Запрос `pg_is_in_recovery()` вернул `false`, что подтверждает работу ноды как `primary`. Создана тестовая таблица `lab_test` и добавлена первая запись.

```
lab1 git:(main) docker exec -e PGPASSWORD=postgres -it pg-master \
psql -h 127.0.0.1 -p 5432 -U postgres -d postgres
psql (15.19 (Debian 15.19-1.pgdg13+2))
Type "help" for help.

postgres=# SELECT pg_is_in_recovery();
pg_is_in_recovery
-----
f
(1 row)

postgres=# CREATE TABLE lab_test (
        id SERIAL PRIMARY KEY,
        message VARCHAR(255)
);
CREATE TABLE
postgres=# INSERT INTO lab_test (message)
VALUES ('hello from master');
INSERT 0 1
postgres=# SELECT * FROM lab_test;
 id |      message
----+-----
  1 | hello from master
(1 row)
```

Рисунок 4 - Создание таблицы `lab_test` и записи на текущем Leader

2.2. Проверка данных на реплике и режима `read-only`

На `pg-slave` запрос `pg_is_in_recovery()` вернул `true`. Таблица `lab_test` и строка `hello from master` появились автоматически. Попытка выполнить `INSERT` непосредственно на `standby` завершилась ошибкой `cannot execute INSERT in a read-only transaction`, что подтверждает корректную работу реплики в режиме только для чтения.

```

lab1 git:(main) docker exec -e PGPASSWORD=postgres -it pg-slave \
psql -h 127.0.0.1 -p 5432 -U postgres -d postgres
psql (15.19 (Debian 15.19-1.pgdg13+2))
Type "help" for help.

postgres=# SELECT pg_is_in_recovery();
pg_is_in_recovery
-----
t
(1 row)

postgres=# SELECT * FROM lab_test;
 id |      message
----+-----
  1 | hello from master
(1 row)

postgres=# INSERT INTO lab_test (message)
VALUES ('hello from slave');
ERROR:  cannot execute INSERT in a read-only transaction
postgres=#

```

Рисунок 5 - Проверка репликации и запрета записи на standby-ноде

3. Часть 3. Настраиваем высокую доступность через HAProxy

3.1. Добавление HAProxy в docker-compose.yml

В Compose-проект добавлен HAProxy, который используется как единая точка подключения к PostgreSQL. На хосте порт 7000 был занят другим процессом, поэтому для страницы статистики использован проброс 7001:7000. Внутри контейнера конфигурация осталась стандартной и HAProxy продолжает слушать 7000.

```

haproxy:
  image: haproxy:3.0
  container_name: postgres_entrypoint
  ports:
    - "5432:5432"
    - "7001:7000"
  depends_on:
    - pg-master
    - pg-slave
    - zoo
  volumes:
    - ./haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg

```

3.2. Конфигурация haproxy.cfg

```
global
    maxconn 100

defaults
    log global
    mode tcp
    retries 3
    timeout client 30m
    timeout connect 4s
    timeout server 30m
    timeout check 5s

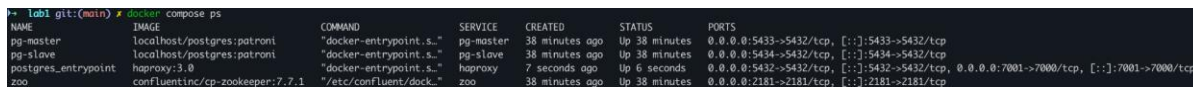
listen stats
    mode http
    bind *:7000
    stats enable
    stats uri /

listen postgres
    bind *:5432
    option httpchk
    http-check expect status 200
    default-server inter 3s fall 3 rise 2 on-marked-down shutdown-sessions
    server postgresql_pg_master_5432 pg-master:5432 maxconn 100 check port 8008
    server postgresql_pg_slave_5432 pg-slave:5432 maxconn 100 check port 8008
```

HAProxy выполняет HTTP health-check REST API Patroni на порту 8008. Для write-entypoint активной считается нода, которая отвечает HTTP 200 как текущий Leader; standby отвечает HTTP 503 и исключается из активных backend.

3.3. Запуск и проверка HAProxy

После добавления сервиса проект был обновлен командой `docker compose up -d`. В результате запущены четыре контейнера: `pg-master`, `pg-slave`, `zoo` и `postgres_entypoint`.



NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
pg-master	localhost/postgres:patroni	"docker-entrypoint.s..."	pg-master	38 minutes ago	Up 38 minutes	0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
pg-slave	localhost/postgres:patroni	"docker-entrypoint.s..."	pg-slave	38 minutes ago	Up 38 minutes	0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp
postgres_entypoint	haproxy:3.0	"docker-entrypoint.s..."	haproxy	7 seconds ago	Up 6 seconds	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp, 0.0.0.0:7000->7000/tcp, [::]:7000->7000/tcp
zoo	confluentinc/cp-zookeeper:7.7.1	"/etc/confluent/dock..."	zoo	38 minutes ago	Up 38 minutes	0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp

Рисунок 6 - Состояние контейнеров после добавления HAProxy

В логах HAProxy зафиксирована успешная загрузка конфигурации. Реплика pg-slave получила HTTP 503 от Patroni API и была исключена из write-backend, при этом один активный сервер остался доступен.

```
lab1 git:(main) # docker compose logs --tail=50 haproxy
postgres_entrypoint | [NOTICE] (1) : New worker (8) forked
postgres_entrypoint | [NOTICE] (1) : Loading success.
postgres_entrypoint | [WARNING] (8) : Server postgres/postgresql.pg_slave_5432 is DOWN, reason: Layer7 wrong status, code: 503, info: "Service Unavailable", check duration: 14ms. 1 active and 0 backup servers left. 0 sessions active, 0 requested, 0 remaining in queue.
```

Рисунок 7 - Логи HAProxy после запуска

На странице статистики HAProxy текущий Leader pg-master отображается как UP с результатом L7OK/200, а pg-slave - как DOWN для данного write-backend с результатом L7STS/503.

Frontend		Backend	
Queue	Session rate	Session rate	Session rate
Cur	Max	Cur	Max
0	0	0	0

pgmaster		pgslave	
Queue	Session rate	Session rate	Session rate
Cur	Max	Cur	Max
0	0	0	0

Рисунок 8 - HAProxy до отказа лидера: pg-master активен, pg-slave исключен из write-backend

3.4. Проверка подключения через единый endpoint

Подключение выполнено через адрес postgres_entrypoint:5432. Запрос pg_is_in_recovery() вернул false, а чтение и запись в lab_test выполнялись успешно. Таким образом, HAProxy направил клиентское соединение на текущий Leader.

```

lab1 git:(main) x docker exec -e PGPASSWORD=postgres -it pg-master \
psql -h postgres_entrypoint -p 5432 -U postgres -d postgres
psql (15.19 (Debian 15.19-1.pgdg13+2))
Type "help" for help.

postgres=# SELECT pg_is_in_recovery();
pg_is_in_recovery
-----
f
(1 row)

postgres=# SELECT * FROM lab_test;
 id |      message
----+-----
  1 | hello from master
(1 row)

postgres=# INSERT INTO lab_test (message)
VALUES ('insert through haproxy');
INSERT 0 1
postgres=# SELECT * FROM lab_test;
 id |      message
----+-----
  1 | hello from master
  2 | insert through haproxy
(2 rows)

postgres=#

```

Рисунок 9 - Чтение и запись через HAProxy до отказа лидера

4. Проверка отказоустойчивости (failover)

4.1. Принудительное отключение текущего лидера

Перед экспериментом еще раз проверено, что pg-master является Leader, а pg-slave - Sync Standby. Затем ведущий контейнер принудительно остановлен командой `docker stop pg-master`.

```

lab1 git:(main) x docker exec -it pg-master patronictl -c /postgres0.yml list
+ Cluster: my_cluster (7693595896378900513) -----+-----+-----+-----+-----+-----+
| Member   | Host       | Role       | State   | TL | Receive LSN | Lag | Replay LSN | Lag |
+-----+-----+-----+-----+-----+-----+-----+-----+
| postgresql0 | pg-master | Leader     | running | 1 |             |     |             |     |
| postgresql1 | pg-slave  | Sync Standby | streaming | 1 | 0/4000238 | 0 | 0/4000238 | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+

```

Рисунок 10 - Состояние кластера непосредственно перед отключением Leader

4.2. Автоматическое переключение Patroni

После остановки pg-master Patroni обнаружил потерю лидера и повысил postgresql1 (pg-slave) до роли Leader. Timeline изменилась с 1 на 2, что соответствует переключению к новой истории WAL.

```
lab1 git:(main) x docker exec -it pg-master patronictl -c /postgres0.yml list
+ Cluster: my_cluster (7693595896378900513) -----+-----+-----+-----+-----+
| Member   | Host   | Role   | State   | TL | Receive LSN | Lag | Replay LSN | Lag |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| postgresql0 | pg-master | Leader | running | 1 |             |     |             |     |
| postgresql1 | pg-slave  | Sync Standby | streaming | 1 | 0/4000238 | 0 | 0/4000238 | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug pg-master
  Learn more at https://docs.docker.com/go/debug-cli/
lab1 git:(main) x docker stop pg-master
pg-master
lab1 git:(main) x docker exec -it pg-slave patronictl -c /postgres1.yml list
+ Cluster: my_cluster (7693595896378900513) -----+-----+-----+-----+-----+
| Member   | Host   | Role   | State   | TL | Receive LSN | Lag | Replay LSN | Lag |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| postgresql1 | pg-slave | Leader | running | 2 |             |     |             |     |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Рисунок 11 - Остановка pg-master и автоматическое повышение pg-slave до Leader

4.3. Реакция HAProxy на failover

HAProxy сначала отметил pg-master как DOWN из-за Layer4 timeout. На короткое время доступных backend-серверов не было. После завершения failover Patroni на pg-slave начал отвечать HTTP 200, и HAProxy автоматически включил эту ноду как активный backend.

```
lab1 git:(main) x docker compose logs --tail=50 haproxy
postgres_entrypoint | [NOTICE] (1) : New worker (8) forked
postgres_entrypoint | [NOTICE] (1) : Loading success.
postgres_entrypoint | [WARNING] (8) : Server postgres/postgresql_pg_slave_5432 is DOWN, reason: Layer7 wrong status, code: 503, info: "Service Unavailable", check duration: 14ms. 1 active and 0 backup servers left. 0 sessions active, 0 requested, 0 remaining in queue.
postgres_entrypoint | [WARNING] (8) : Server postgres/postgresql_pg_master_5432 is DOWN, reason: Layer4 timeout, check duration: 3006ms. 0 active and 0 backup servers left. 0 sessions active, 0 requested, 0 remaining in queue.
postgres_entrypoint | [ALERT] (8) : proxy 'postgres' has no server available!
postgres_entrypoint | [WARNING] (8) : Server postgres/postgresql_pg_slave_5432 is UP, reason: Layer7 check passed, code: 200, check duration: 4ms. 1 active and 0 backup servers online. 0 sessions requested, 0 total in queue.
```

Рисунок 12 - Логи HAProxy во время переключения лидера

4.4. Проверка чтения и записи после failover

Подключение к базе выполнено через тот же адрес postgres_entrypoint:5432, но клиент запущен из оставшегося контейнера pg-slave. Запрос pg_is_in_recovery() снова вернул false. Старые данные сохранились, а INSERT новой строки insert after failover завершился успешно. Это подтверждает сохранение работоспособности клиента после автоматического переключения.

```

lab1 git:(main) x docker exec -e PGPASSWORD=postgres -it pg-slave \
psql -h postgres_entrypoint -p 5432 -U postgres -d postgres
psql (15.19 (Debian 15.19-1.pgdg13+2))
Type "help" for help.

postgres=# SELECT pg_is_in_recovery();
pg_is_in_recovery
-----
f
(1 row)

postgres=# SELECT * FROM lab_test;
 id |      message
----+-----
  1 | hello from master
  2 | insert through haproxy
(2 rows)

postgres=# INSERT INTO lab_test (message)
VALUES ('insert after failover');
INSERT 0 1
postgres=# SELECT * FROM lab_test;
 id |      message
----+-----
  1 | hello from master
  2 | insert through haproxy
 35 | insert after failover
(3 rows)

postgres=#

```

Рисунок 13 - Чтение и запись через тот же HAProxy endpoint после failover

4.5. Возврат ранее отключенной ноды

После запуска pg-master он не стал отбирать роль Leader у работающего pg-slave, а присоединился к кластеру как Sync Standby. Состояние streaming и Lag = 0 показывают, что реплика полностью синхронизировалась с новым лидером.

```

lab1 git:(main) x docker start pg-master
pg-master
lab1 git:(main) x docker exec -it pg-master patronictl -c /postgres0.yml list
+ Cluster: my_cluster (7693595896378900513) -----+-----+
| Member | Host | Role | State | TL | Receive LSN | Lag | Replay LSN | Lag |
+-----+-----+-----+-----+-----+-----+-----+-----+
| postgresql0 | pg-master | Sync Standby | streaming | 2 | 0/5003490 | 0 | 0/5003490 | 0 |
| postgresql1 | pg-slave | Leader | running | 2 | | | | |
+-----+-----+-----+-----+-----+-----+-----+-----+

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug pg-master
Learn more at https://docs.docker.com/go/debug-cli/
lab1 git:(main) x

```

Рисунок 14 - Состояние Patroni после возврата pg-master

На вернувшейся ноте pg-master присутствует строка insert after failover, которая была записана в период ее отсутствия. Следовательно, нода автоматически получила изменения после возврата в кластер.

```
lab1 git:(main) x docker start pg-master
pg-master
lab1 git:(main) x docker exec -it pg-master patronictl -c /postgres0.yml list
+ Cluster: my_cluster (7693595896378900513) -----+-----+-----+-----+
| Member | Host | Role | State | TL | Receive LSN | Lag | Replay LSN | Lag |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| postgresql0 | pg-master | Sync Standby | streaming | 2 | 0/5003490 | 0 | 0/5003490 | 0 |
| postgresql1 | pg-slave | Leader | running | 2 | | | | |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug pg-master
Learn more at https://docs.docker.com/go/debug-cli/
lab1 git:(main) x docker exec -e PGPASSWORD=postgres -it pg-master \
psql -h 127.0.0.1 -p 5432 -U postgres -d postgres
psql (15.19 (Debian 15.19-1.pgdg13+2))
Type "help" for help.

postgres=# SELECT * FROM lab_test;
 id | message
-----+-----
  1 | hello from master
  2 | insert through haproxy
 35 | insert after failover
(3 rows)

postgres=#
```

Рисунок 15 - Данные на вернувшейся ноте: запись, созданная во время ее отсутствия, синхронизирована

Финальная страница статистики HAProxy показывает обратную исходной картину: pg-slave, который стал новым Leader, отвечает HTTP 200 и имеет статус UP; pg-master после возврата работает как standby и для write-endpoint получает HTTP 503.

state		Queue		Session rate		Sessions		Bytes		Denied		Errors		Warnings		Status		Server	
		Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Cur	Max	Cur	Max
Frontend		0	2		2	3	100	4	1	278	40	327	0	0	1	0	0	0	0
Backend		0	0		0	0	10	0	0	0	0	0	0	0	0	0	0	0	0

postgres		Queue		Session rate		Sessions		Bytes		Denied		Errors		Warnings		Status		Server	
		Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Cur	Max	Cur	Max
Frontend		0	1		0	1	100	1	1	12m38s	423	597	0	0	0	0	0	0	0
postgresql_pg_master_5432		0	0		0	1	100	1	1	12m38s	423	597	0	0	0	0	0	0	0
postgresql_pg_slave_5432		0	0		0	1	100	1	1	5m24s	422	1,013	0	0	0	0	0	0	0
Backend		0	0		0	1	10	2	2	5m24s	845	1,950	0	0	0	0	0	0	0

Рисунок 16 - Финальное состояние HAProxy после failover и возврата старой ноды

4.6. Анализ произошедшего

После принудительного отключения Leader Patroni перестал получать его heartbeat через ZooKeeper. После истечения времени обнаружения отказа доступная синхронная реплика была повышена до Leader. HAProxy обнаружил потерю старого backend, а затем автоматически переключил новые соединения

на pg-slave после того, как REST API Patroni на этой ноде начал отвечать HTTP 200. Адрес клиентского подключения при этом не изменился. После возврата pg-master он присоединился в роли standby и догнал новый Leader по WAL.

5. Вывод

В ходе лабораторной работы был развернут высокодоступный кластер PostgreSQL на базе Docker, Patroni, ZooKeeper и HAProxy. Были настроены и проверены синхронная потоковая репликация, автоматическое определение лидера и единая точка клиентского подключения.

Эксперимент с принудительным отключением pg-master показал работоспособность автоматического failover: pg-slave был повышен до Leader, HAProxy переключил трафик на новую ведущую ноду, а чтение и запись через прежний endpoint продолжили работать. После возврата pg-master нода автоматически присоединилась как Sync Standby и получила изменения, записанные во время ее отсутствия. Таким образом, цель лабораторной работы достигнута, а требуемый сценарий отказоустойчивости подтвержден практически.